Лабораторна робота №1. Налаштування середовища розробки, проектування архітектури індивідуального проекту та робота з типами даних .NET

Мета: Ознайомитися з архітектурою платформи .NET, навчитися налаштовувати середовище розробки (IDE), опанувати структуру рішень (Solution) та проектів (Project), а також спроектувати базову доменну модель індивідуального проекту.

Загальне завдання:

1. Встановити та налаштувати середовище розробки (Visual Studio, JetBrains Rider або VS Code).
2. Створити нове рішення (Solution) з назвою Project_[Ваше_Прізвище].
3. Додати проект типу Class Library з назвою Core. Цей проект буде містити бізнес-логіку.
4. Додати проект типу Console App з назвою ConsoleUI.
5. Налаштувати залежність: проект ConsoleUI повинен мати посилання (Project Reference) на проект Core.
6. У проекті ConsoleUI (файл Program.cs) реалізувати вивід інформації про поточну версію ОС та обсяг оперативної пам'яті, що використовується додатком (клас System.Environment та System.GC).

Індивідуальне завдання:

1. Обрати тему індивідуального проєкту відповідно до Додатку А, описати основні компоненти, які необхідно буде реалізувати у наступних лабораторних роботах за обраною темою.
2. Згідно з обраною темою (варіантом), у проекті Core створити мінімум 3 класи, які описують предметну область.
3. Кожен клас повинен містити мінімум 3 властивості (Properties) з різними типами даних (string, int, double, DateTime, bool).

4. У проекті ConsoleUI створити по одному екземпляру кожного класу, ініціалізувати їх тестовими даними та вивести опис об'єктів у консоль.

Вибір та підготовка середовища розробки

Студент має право обрати одне з середовищ розробки:

- Visual Studio 2022 Community: оптимально для Windows, необхідно обрати пакет ".NET desktop development" в процесі установки.
- JetBrains Rider: Кросплатформенна IDE з потужним аналізом коду, потребує встановлення .NET SDK.
- VS Code: легкий редактор, що потребує встановлення розширення "C# Dev Kit".

Теоретичні відомості:

Платформа .NET базується на концепції самодокументованого коду. Кожна збірка (.dll або .exe) містить метадані, які дозволяють середовищу виконання (CLR) отримувати повну інформацію про структуру класів.

Основні поняття:

- **Solution (.sln)** — файл-контейнер, що об’єднує кілька проектів для спільної розробки.
- **Project (.csproj)** — окрема одиниця компіляції (бібліотека класів або додаток).
- **Common Type System (CTS)** — стандарт, що визначає правила оголошення та використання типів даних у .NET.

Типи даних поділяються на:

1. **Value Types (Типи значень):** зберігаються в стеку (Stack). Сюди належать int, double, bool, struct, enum. При копіюванні створюється дублікат значення.
2. **Reference Types (Типи посилань):** зберігаються в купі (Heap), а в стеку міститься лише адреса (посилання). Сюди належать class, string, interface. При копіюванні копіюється лише посилання на той самий об'єкт.

Контрольні питання:

1. У чому полягає різниця між файлами .sln та .csproj?

2. Яка роль JIT-компілятора у виконанні C#-коду?

3. Чим відрізняється .NET SDK від .NET Runtime? Поясніть з точки зору розробника.

4. Чому для інженерних рішень важливо відокремлювати Core (логіку) від UI (інтерфейсу)?

5. Які переваги надає використання властивостей (Properties) замість публічних полів (Fields)?

6. Що таке "керований код" (Managed Code) і які його основні переваги?

ДОДАТОК А.

Перелік тем індивідуальних завдань для виконання в рамках лабораторних робіт

1. Менеджер завдань (ToDo App) – користувач створює, редагує, виконує та видаляє завдання.
2. Система нотаток – збереження нотаток із можливістю пошуку та категоризації.
3. Онлайн-бібліотека – книги, автори, користувачі з можливістю бронювання.
4. Міні-CRM для клієнтів – управління клієнтами, контактами, угодами.
5. Система для відгуків – користувачі можуть залишати відгуки про товари/послуги.
6. Менеджер подій – створення івентів, реєстрація користувачів, список відвідувачів.
7. Онлайн-магазин (спрощений) – товари, категорії, кошик користувача.
8. Форум або мікроблог – пости, коментарі, лайки.
9. Система для бронювання кімнат/ресурсів – наприклад, переговорних кімнат.
10. API для кулінарних рецептів – рецепти, інгредієнти, категорії.
11. Трекер витрат – доходи, витрати, категорії, підрахунок балансу.
12. Система для голосування/опитувань – створення опитувань та збір голосів.
13. API для кінотеатру – фільми, сеанси, бронювання квитків.
14. Міні-ERP для складу – товари, прихід/витрата, залишки на складі.
15. Система управління курсами – курси, викладачі, студенти, реєстрація.
16. Трекер фізичної активності – тренування, калорії, прогрес.

17. API для подорожей – країни, міста, користувацькі маршрути.

18. Онлайн-довідник лікарів – лікарі, пацієнти, прийоми.

19. Система управління проєктами – проєкти, задачі, виконавці.

20. API для музичної бібліотеки – виконавці, альбоми, треки.

21. Каталог фільмів з рейтингами – пошук, відгуки, рейтинги.

22. Система бронювання столиків у ресторані – столи, бронювання, клієнти.

23. API для інтернет-щоденника – записи за датами, теги.

24. Система обліку техніки в компанії – обладнання, видача, повернення.

25. Онлайн-дошка оголошень – оголошення, категорії, користувачі.

26. Система керування підписками – користувачі підписуються на різні плани, автоматичне продовження підписки, історія платежів.

27. API для навчальної платформи з тестами – створення курсів, завдань, перевірка відповідей, підрахунок балів.

28. Система відстеження замовлень доставки – користувачі створюють замовлення, кур'єри оновлюють статус.

29. Онлайн-конструктор опитувань – створення опитувань, питання, варіанти відповідей, статистика відповідей.

30. Сервіс рекомендацій книг/фільмів – користувачі додають вподобані, отримують персоналізовані рекомендації.

31. Менеджер контактів – збереження контактів, групування за категоріями, пошук та імпорт/експорт.

32. API для управління подіями у спортзалі – розклад тренувань, реєстрація на заняття, ліміти учасників.

33. Система моніторингу серверів/сервісів – додавання серверів, відстеження стану, логування помилок.

34. API для онлайн-ігрового рейтингу – гравці, ігри, рейтинг, таблиця лідерів.

35. Міні-соціальна мережа для обміну фото – користувачі додають фото, лайки, коментарі, теги.

36. Система керування бібліотекою книг – користувачі завантажують, описують та категоризують книги, можливість перегляду статистики читачів.

37. Онлайн-трекер звичок – створення щоденних звичок (наприклад, випити воду, зробити зарядку), відмітка виконання, статистика успішності.

38. API для погодних даних – збереження локальної історії прогнозів та температур, інтеграція з відкритими API.

39. Система управління бібліотекою фото – завантаження, тегування, пошук і завантаження зображень користувачем.

40. Онлайн-планувальник навчання – створення розкладу занять, нагадування, відстеження виконання навчальних завдань.

41. Сервіс управління чергами – користувачі займають місце в черзі, адміністратор керує обслуговуванням (наприклад, для банку чи поліклініки).

42. API для блог-платформи з markdown – користувачі створюють пости у форматі Markdown, підтримка тегів та категорій.

43. Система збереження паролів (Password Manager) – користувачі зберігають свої паролі у зашифрованому вигляді.

44. Онлайн-журнал для викладачів – ведення списку студентів, оцінки, відвідуваність, звіти.

45. Сервіс для управління волонтерськими проєктами – реєстрація проєктів, волонтерів, відстеження участі та результатів.

ДОДАТОК Б.

Приклад оформлення звіту

Олексій Горносталь (Ім’я та прізвище)

КН-117а (номер групи)

Проєктування серверних застосунків (Назва курсу)

Звіт з лабораторної роботи №1 (Номер роботи)

Тема: Dynamic memory management (Тема роботи)

Перевірив: доц., PhD, Віктор Челак
(посада, науковий ступінь, Ім’я та прізвище викладача, який перевіряє роботу)

1. Теоретичний блок

(Невеликий блок того, що використовувалася у завданнях)

Під час роботи з динамічною пам'яттю в сучасних операційних системах можуть виникнути наступні поширені проблеми…

2. Хід виконання

(Опис виконаних дій, розробленої програми та зображення)

На першому кроці… Нижче наведена фрагмент коду програми 1, яка вирішує проблему…

Важливий фрагмент коду з описом

На рис. 1 представлені результати роботи розробленої програми …

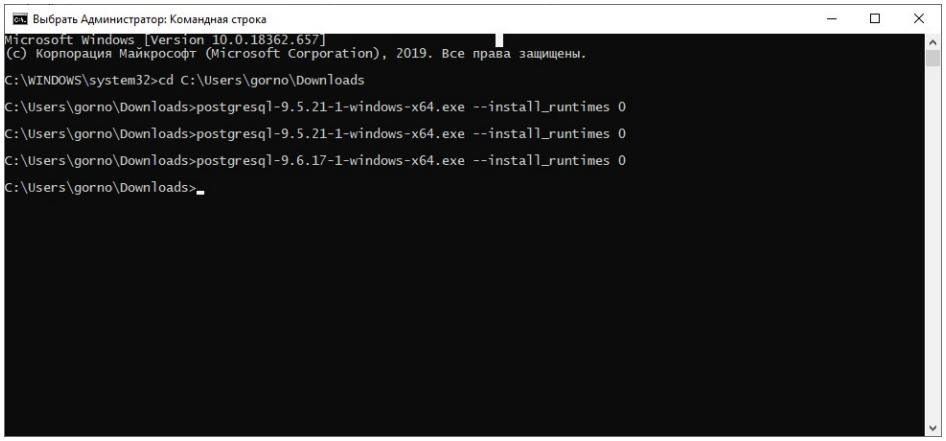

Рисунок 1 – Результат виконання програми

3. Висновки

(Декілька слів з мети та короткий опис конкретних результатів роботи)

У цій лабораторній роботі ми дослідили деякі поширені проблеми, які можуть виникнути під час роботи з динамічною пам'яттю. Ми створили кілька програм, які показували ці помилки та досліджували результати їх виконання.